

Task 1: Processing Time Prediction - Data Preprocessing Documentation

This document outlines the comprehensive data preprocessing pipeline developed for predicting processing times in the government service booking system. Our approach combines systematic data cleaning, advanced feature engineering, and robust outlier handling to achieve optimal model performance.

1. Data Preprocessing

Empty Row Detection and Removal

Why Was This Needed?

- During initial data exploration, missing values were found in a few columns.
- In total, 6092 out of 203693 records had missing or empty values in these columns.

```
Missing values per column:
booking_id           0
citizen_id           0
booking_date         0
appointment_date     0
appointment_time     0
check_in_time        6092
check_out_time       6092
task_id              0
num_documents        0
queue_number         0
satisfaction_rating  0
```

Why Remove These Rows?

- The target variable (processing time) cannot be calculated without both timestamps.
- Keeping incomplete records would introduce errors and reduce model reliability.
- Removing these rows ensures that all data used for modeling is valid and complete.

Impact of Cleaning

- **Original dataset size:** 203,693 rows
- **After cleaning:** 197601 rows
- **Data retention:** 97.01%
- Minimal data loss, but significant improvement in data quality and model trustworthiness.

```
Original dataset shape: (203693, 11)
Cleaned dataset shape: (197601, 11)
Rows removed: 6092
Percentage of data retained: 97.01%
```

Outlier Detection and Handling

Why Was This Needed?

- Exploratory analysis revealed extreme values in processing times.
- Such extreme values are unrealistic for the business context and can negatively impact model training.

IMPACT ON PROCESSING TIMES:						
	Mean	Std	Min	Max	Skewness	
Original	48.8	24.0	5.9	217.6	1.46	
IQR Cleaned	47.9	22.6	5.9	154.9	1.21	

Why Use Task-Based Outlier Removal?

- Different tasks naturally have different expected processing times.
- A global outlier threshold would remove valid data for some tasks and miss true outliers for others.
- Task-based outlier handling uses statistical boundaries (IQR method) for each task type, ensuring only truly anomalous records are removed.

✍ APPLYING IQR METHOD:				
Task	Original	Removed	Lower_Bound	Upper_Bound
TASK-001	10706	172	26.4	83.2
TASK-002	5239	89	14.2	120.9
TASK-003	6901	175	10.0	111.0
TASK-004	9516	148	12.3	41.6
TASK-005	16258	246	16.8	53.8
TASK-006	6103	174	5.0	76.0
TASK-007	12675	300	3.5	43.5
TASK-008	12267	246	5.5	53.1
TASK-009	7616	124	21.0	59.7
TASK-010	10028	176	14.5	59.6
TASK-011	10902	163	16.4	46.9
TASK-012	5472	91	18.5	100.0
TASK-013	14378	377	5.6	154.9
TASK-014	12602	190	19.7	59.5
TASK-015	13910	302	11.0	137.5
TASK-016	12435	151	25.2	71.3
TASK-017	10741	200	19.9	132.1
TASK-018	7804	170	9.2	49.7
TASK-019	12048	221	17.5	95.8

Impact of Outlier Handling

- Data retention after outlier removal: ~98%

```
📋 TASK-WISE OUTLIER REMOVAL SUMMARY:
TASK-001: 172/10706 removed (1.6%)
TASK-002: 89/5239 removed (1.7%)
TASK-003: 175/6901 removed (2.5%)
TASK-004: 148/9516 removed (1.6%)
TASK-005: 246/16258 removed (1.5%)
TASK-006: 174/6103 removed (2.9%)
TASK-007: 300/12675 removed (2.4%)
TASK-008: 246/12267 removed (2.0%)
TASK-009: 124/7616 removed (1.6%)
TASK-010: 176/10028 removed (1.8%)
TASK-011: 163/10902 removed (1.5%)
TASK-012: 91/5472 removed (1.7%)
TASK-013: 377/14378 removed (2.6%)
TASK-014: 190/12602 removed (1.5%)
TASK-015: 302/13910 removed (2.2%)
TASK-016: 151/12435 removed (1.2%)
TASK-017: 200/10741 removed (1.9%)
TASK-018: 170/7804 removed (2.2%)
TASK-019: 221/12048 removed (1.8%)

Total outliers removed across all tasks: 3715
```

- The cleaned data is more realistic, less skewed, and better suited for accurate modeling.

Handling Categorical Features

- **Encoding Strategy:**

We applied One-Hot Encoding (OHE) to categorical variables (task_id, task_hour, and section_id).

- **Rationale:**

These features are nominal categories with no intrinsic ordering (e.g., task IDs are identifiers, not ordinal values).

Using label encoding could incorrectly impose an ordinal relationship, which may mislead the model.

One-hot encoding ensures that each category is represented independently, avoiding artificial hierarchies.

- **Implementation Detail:**

For each categorical feature, a separate binary column was created for each unique category.

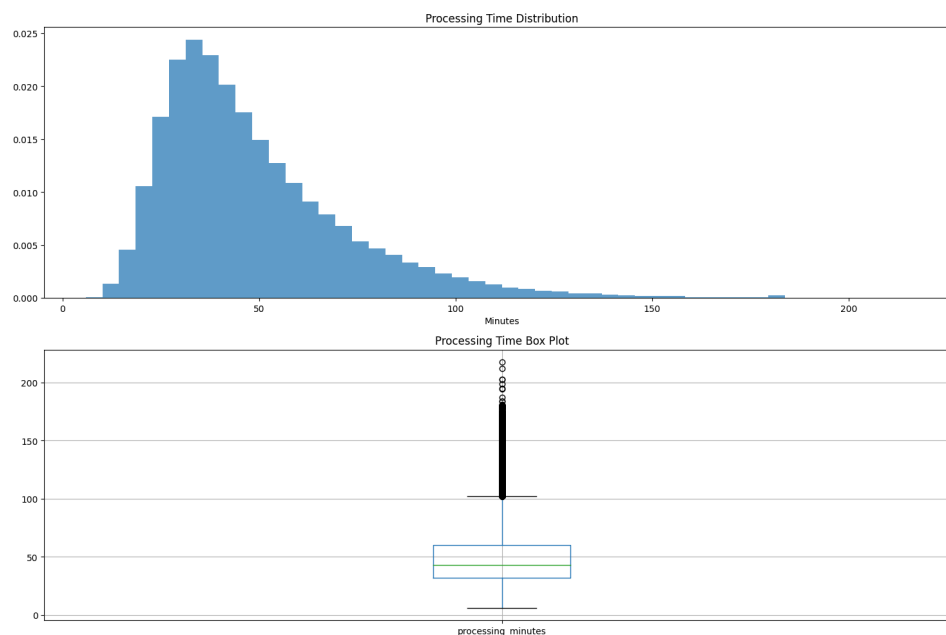
This representation allows models (particularly linear or tree-based ones) to learn category-specific effects without introducing unintended bias.

2. Exploratory Data Analysis (EDA)

2.1 Target Variable Analysis

The target variable ``processing_minutes`` represents the time difference between check-out and check-in times.

- Distribution: Highly right-skewed with significant outliers
- Range: 0 to several hundred minutes
- Outliers: Identified using task-specific IQR method due to varying processing time patterns across different tasks



- Rationale: Different tasks have inherently different processing time patterns. A global outlier threshold would incorrectly classify normal processing times for complex tasks as outliers.

2.2 Temporal Pattern Analysis

2.2.1 Time-of-Day Patterns

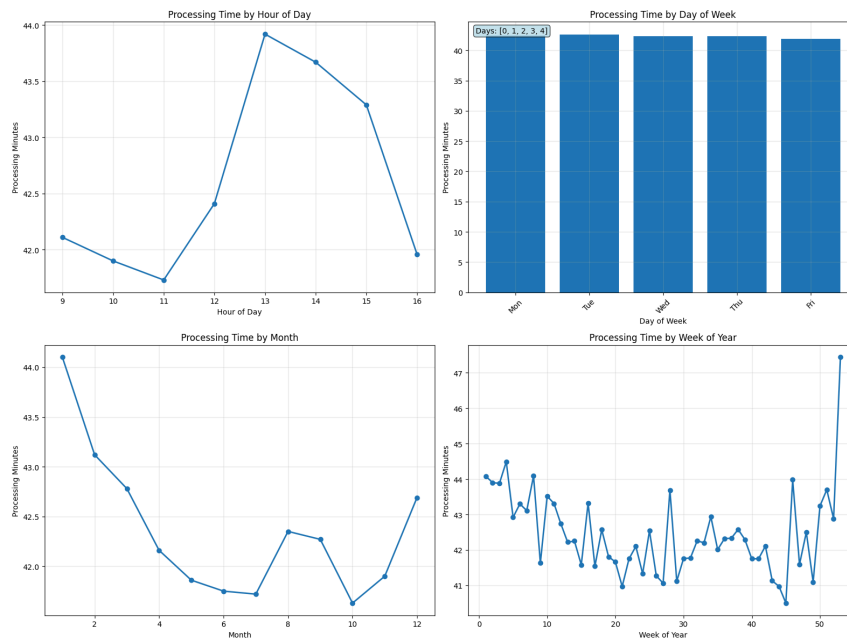
- Peak Hours : Processing times vary by hour of day, but not significantly
- Patterns: At rush hours like morning and lunch, processing time increases a bit. Not a significant increase.

2.2.2 Day-of-Week Effects

- Weekly Patterns: Processing times show distinct patterns across weekdays
- Volume Impact: Higher volume days correlate with longer processing times
- Seasonal Effects: Month-to-month variations observed

2.2.3 Time Series Analysis

- Trend Analysis: Slight temporal trends in processing times over the dataset period
Trend Analysis: Slope: -0.0077 minutes/day
- Autocorrelation: Limited autocorrelation suggests daily patterns are more important than long-term trends
- Stationarity: Relatively stationary series with cyclical components



2.3 Categorical Variable Analysis

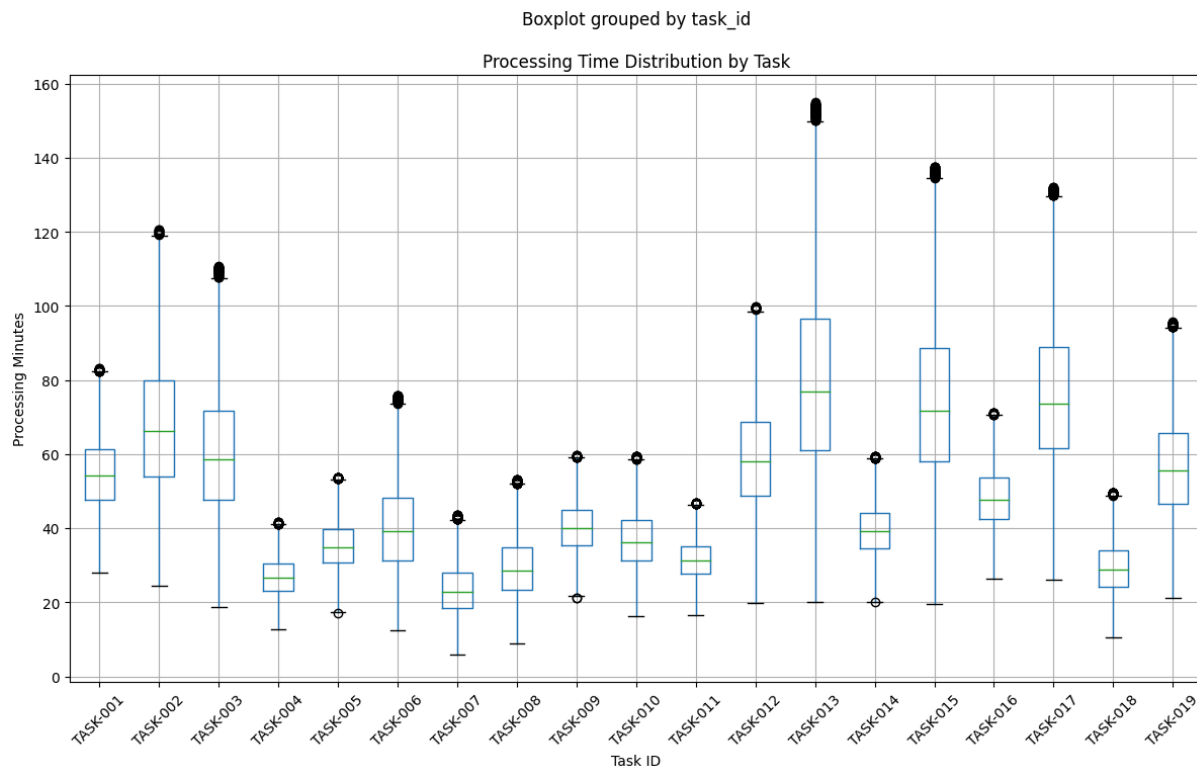
2.3.1 Task ID Analysis

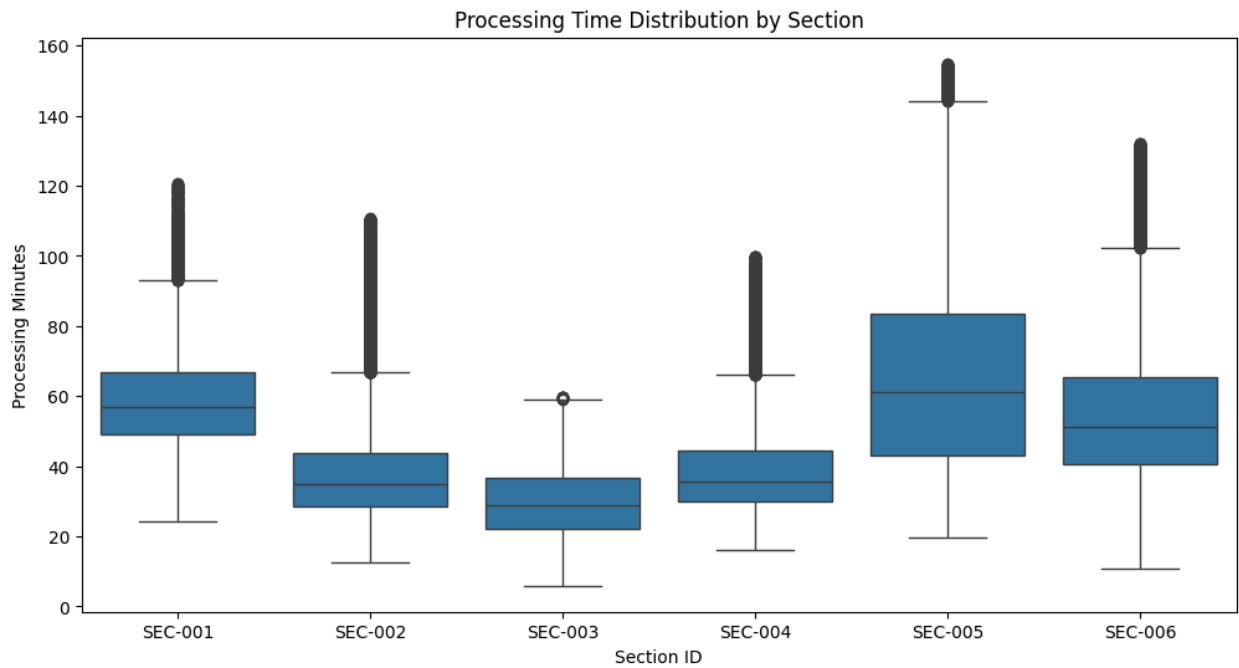
- Primary Predictor: Task ID shows the strongest relationship with processing times
- Variance Explained: Task ID explains ~60% of processing time variance
- Distinct Patterns: Each task has characteristic processing time distributions

For feature engineering purposes, we joined tasks to get section_Id.

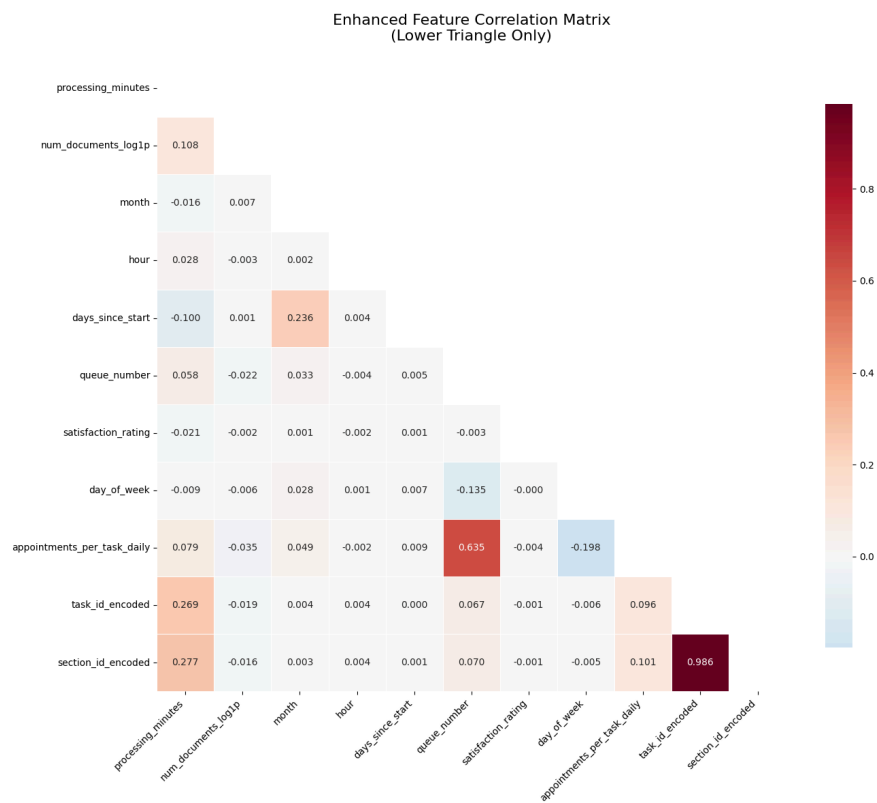
2.3.2 Section ID Analysis

- Secondary Predictor: Section ID provides additional predictive power
- Hierarchical Relationship: Tasks belong to sections, creating natural hierarchy
- Complementary Information: Adds value beyond task-specific patterns





2.4 Correlation Analysis



The correlation heatmap confirmed where redundancy existed and guided us to prioritize task-level features (`task_id`) and workload-related metrics while dropping overlapping variables. Low-correlation time variables were transformed into cyclical encodings to capture patterns not visible in simple correlations.

3. Feature Engineering

After performing EDA and correlation analysis, we prioritized features based on predictive power, interpretability, and redundancy avoidance. The final feature groups are:

1. Categorical Features

- **task_id (Primary priority)**
 - Directly represents the type of task, which has the strongest influence on processing time.
 - Rationale: From correlation analysis, `task_id` showed a meaningful relationship with the target.
- **task_hour (Task × Time interaction)**
 - A composite feature combining task type and hour of the day.
 - Rationale: Captures task-specific temporal patterns. For example, some tasks may consistently take longer in the morning vs afternoon.
- **section_id**
 - Represents the section handling the task.
 - Rationale: Retained at lower priority because of high correlation with `task_id`, but still included for models that can handle redundancy (tree-based models), in case section-level variations exist.

2. Numeric Features

Base Numeric

- **num_documents_log1p**
 - Log-transformed count of documents.

- Rationale: Strong domain relevance since more documents directly increase processing load. Log transform stabilizes variance and reduces skewness.

Time Numerics

- **dow_sin, dow_cos**
 - Cyclical encoding of day-of-week.
- **month_sin, month_cos**
 - Cyclical encoding of month.
- **days_since_start**
 - Linear time trend feature.
 - Rationale: Time features capture weekly/monthly seasonality and long-term drifts in workload patterns. Using sine/cosine encodings ensures smooth representation of cyclical variables (e.g., Monday and Sunday are close, not far apart).

Volume Features

- **daily_section_volume**: Total volume handled by section per day.
- **hourly_section_volume**: Section load at hourly granularity.
- **daily_task_volume**: Total tasks of a type per day.
- **volume_rollmean7**: 7-day rolling average workload.
- **volume_pct_rank**: Rank of workload volume relative to history.

Rationale: These capture workload intensity, congestion effects, and recent trends. High volumes are often linked with longer processing times.

Why We Selected These Features?

- Prioritized task-level features (task_id, task_hour) as the most direct drivers of processing time.
- Retained document volume and workload metrics since they capture the operational burden on the system.
- Applied cyclical encodings for time-based variables to preserve continuity and periodicity.
- Dropped redundant or low-signal features identified in correlation analysis (e.g., satisfaction rating, raw hour/month, redundant section/task encodings).

Feature Selection Process

Step 1: Feasibility & Leakage Screening

(Keep only features available at prediction time; drop leakage signals)



Step 2: Univariate Signal & Business Relevance

(EDA shortlist; validate with distributions & domain sense)



Step 3: Multicollinearity Control

(Use cyclical encodings; remove redundant workload features)



Step 4: Interaction Exploration

(Evaluate feature interactions; retain task_hour)



Step 5: Cross-Validation & Ablation

(Time split + CV; keep features improving RMSE/MAE consistently)



Step 6: Simplicity vs Performance

(Choose smallest effective feature set; ensure robustness)

Train–Inference Parity and Missing Data Strategy

- **Feature scope at inference**

Uses only date, time, task_id, and the public task map (tasks.csv).

Excludes staffing-related features to avoid data leakage.

- **Date & time parsing**

appointment_date converted with pd.to_datetime.

appointment_time normalized to HH:MM:SS; accepts flexible formats such as "0900" or "09:00".

- **Numeric features**

Imputed using SimpleImputer(strategy="mean").

num_documents transformed with log1p.

→ Task-level mean imputation (from training data) applied first.

→ Falls back to global mean if task-level mean is unavailable.

- **Categorical features**

Encoded with OneHotEncoder(handle_unknown="ignore").

Columns encoded: task_id, task_hour, section_id.

Unseen categories at inference map to all-zero vectors.

Rare missing values also propagate to all-zero vectors.

- **Schema consistency**

Inference builder enforces presence of all categorical_cols + numeric_cols.

Missing columns are created as NaN before applying the fitted preprocessor, ensuring alignment with training schema.

- **Workload aggregate features**

daily_section_volume, hourly_section_volume, and daily_task_volume computed from same-day/hour data only.

volume_rollmean7 calculated using past section data with min_periods=1 (early days use shorter histories).

No forward-looking information (no leakage).

- **Joins with task map**

Left join performed with tasks.csv.

If task_id is missing, section_id becomes NaN but processing continues since handle_unknown="ignore" is active.

Such cases are logged for traceability.

- **Reproducibility**

Fixed random seed (RNG_SEED).

Preprocessor and model saved together with joblib to ensure consistent encodings at inference.